

AVICENA

Healthcare SaaS Platform

Project Documentation

Prepared by: ENG. Mohamed Ali

Role: Full-Stack Developer

Repository: github.com/Mohamedali409/avicena-platform

Date: July 2026

Status: Active Development (MVP Phase)

Table of Contents

1. Project Idea and Objectives

Avicena is a full-stack Healthcare SaaS (Software-as-a-Service) platform that connects patients, doctors, and laboratories inside a single digital ecosystem. The name is inspired by Ibn Sina (Avicenna), the medieval physician and philosopher, reflecting the project's ambition to bring a modern, structured, and intelligent approach to healthcare delivery. The platform digitizes core clinical workflows — booking, consultation, diagnosis, and follow-up — through a unified, role-based web application backed by real-time communication and a scalable service architecture.

The system is built as a modular monolith on Node.js and MongoDB, structured so that individual modules (appointments, chat, video, labs, subscriptions, etc.) can later be extracted into independent microservices as the platform grows. This design decision reflects a production-oriented mindset rather than a simple prototype.

1.1 Objectives

- Provide a single digital platform where patients can discover doctors, book appointments, and manage their medical journey end-to-end.
- Enable real-time, secure communication (text chat and video/audio calls) between patients and doctors without relying on third-party tools.
- Digitize medical records so consultation history, diagnoses, and prescriptions are stored, retrievable, and auditable.
- Integrate laboratories and diagnostic centers into the same ecosystem so lab results feed directly into a patient's medical record.
- Introduce a subscription-based (SaaS) revenue model with tiered feature access (Free / Basic / Premium).
- Build an architecture that can scale from an MVP to a production telemedicine system, including future AI-assisted features (symptom triage chatbot, smart recommendations).
- Apply Role-Based Access Control (RBAC) so Patients, Doctors, Admins, and Labs each operate within clearly separated permissions and views.

2. Problem Statement

Access to healthcare is frequently slowed down by fragmented, manual, and non-digital processes. Patients often struggle to find the right specialist, book an appointment without phone calls or in-person visits, or keep a consolidated history of their consultations, prescriptions, and lab results. On the other side, doctors and clinics lack a lightweight, affordable system to manage their schedules, patient records, and communication without relying on multiple disconnected tools (spreadsheets, WhatsApp, paper records).

This fragmentation creates several concrete problems that Avicena directly addresses:

- No unified channel for patients to discover, evaluate, and book appointments with doctors based on specialization and availability.
- Medical history is scattered across paper files or disconnected systems, making it hard for a doctor to review a patient's past complaints, diagnoses, and treatments.
- Remote consultations typically depend on third-party video/chat tools that are not integrated with the medical record or the appointment system, breaking the workflow and losing context.
- Laboratories operate as isolated entities; there is no streamlined way for a doctor to view a patient's lab results directly inside the same platform used for consultations.
- Small clinics and independent doctors often cannot afford enterprise-grade Health Information Systems (HIS), leaving a gap for an accessible, subscription-based SaaS alternative.

Avicena's core value proposition is closing this gap: a single, affordable, and extensible SaaS platform that keeps the patient, doctor, and lab data connected in one place, with real-time communication built directly into the clinical workflow.

3. System Overview and Features

Avicena is organized around four primary actors — Patient, Doctor, Admin, and Lab — each interacting with a shared backend through role-specific REST endpoints and, for real-time features, Socket.IO channels. The current codebase implements the backend (Node.js / Express / MongoDB) as the core of the MVP, with the frontend consuming the same REST/WebSocket API.

3.1 System Actors

- Patient (User): registers, books appointments, consults doctors, views medical history, and manages a subscription.
- Doctor: manages availability, handles appointments and consultations, writes medical reports, and communicates with patients.
- Admin: oversees the platform — manages doctors, users, appointments, consultations, reports, and lab verification.
- Lab: manages its profile, available tests, pricing, and working hours; results integrate with the patient's medical record.

3.2 Core Features

Authentication & Access Control

- Separate login flows for Patient, Doctor, Admin, and Lab, each issuing a scoped JWT.
- Role-Based Access Control (RBAC) enforced through dedicated Express middleware guards (authGuard, doctorGuard, adminGuard, labGuard).

Appointment System

- Book, cancel, and track appointments with conflict-aware slot management (Doctor.slots_booked).
- Doctor-side and Admin-side controls to complete or cancel appointments.

Real-Time Chat

- Patients send a chat request to a doctor; the doctor can accept or reject it before a conversation opens.
- Socket.IO-based messaging with read/unread tracking and persisted conversation history in MongoDB.

Video / Audio Consultation

- WebRTC-based calls (via simple-peer) with Socket.IO used purely for signaling.
- Call sessions are logged (caller/receiver, type, status, duration) and linked to a consultation.

Medical Records & Reports

- Doctors record complaint, examination, diagnosis, treatment plan, and next-visit date for every consultation.
- Reports are retrievable by both the doctor and the patient, and exportable as PDF via PDFKit.

Labs & Radiology Integration

- Labs maintain a catalog of tests with price and duration, working hours, and a verification flag controlled by Admin.

Subscription System (SaaS Model)

- Free / Basic / Premium plans with feature flags (video call access, monthly consultation limits, priority support).
- Subscription lifecycle: active, expired, cancelled — with history tracking per patient.

Notifications

- A unified notification model covering appointments, consultations, reports, payments, chat, and system events, per recipient type (user, doctor, admin, lab).

Admin Dashboard

- Platform-wide visibility: manage doctors, users, appointments, consultations, and reports from a single control surface.

Planned / Future Features

- AI-powered medical assistant chatbot for basic symptom guidance (OpenAI integration already present as a dependency).
- Payment gateway integration for real subscription billing.
- Mobile application (React Native) and advanced analytics.

4. Technologies Used

The technology choices reflect a modern JavaScript/Node.js stack optimized for real-time features, horizontal scalability, and rapid iteration.

Layer	Technology / Library	Purpose
Backend Runtime	Node.js + Express.js 5	Core REST API server and routing
Database	MongoDB + Mongoose	Primary document data store & ODM/schema layer
Real-Time Communication	Socket.IO	Live chat messaging and WebRTC signaling
Video/Audio Calls	WebRTC (simple-peer)	Peer-to-peer video/audio consultations
Cache & Queue	Redis (ioredis) + BullMQ	Presence tracking, caching, and background jobs
Authentication	JSON Web Tokens (jsonwebtoken) + bcrypt/bcryptjs	Stateless auth and password hashing
Validation	Zod + validator.js	Request/schema validation
File & Media Storage	Cloudinary	Profile images and uploaded media
Email Service	Nodemailer	Transactional and notification emails
PDF Generation	PDFKit	Exportable medical report documents
AI Integration (planned)	OpenAI SDK	Future medical assistant chatbot
Security Middleware	Helmet, CORS, express-rate-limit	HTTP hardening and abuse protection
Logging	Morgan	HTTP request logging
Containerization	Docker + Docker Compose	Local/prod orchestration of API, MongoDB, and Redis
API Documentation	Swagger (OpenAPI) + Postman Collection	Endpoint documentation and manual testing
Tooling	ESLint, Prettier, Nodemon	Code quality and developer experience

5. System Architecture / Workflow

Avicena follows a Modular Monolith architecture guided by Clean Architecture principles: the codebase is organized into self-contained domain modules (auth, users, doctors, appointments, chat, video-call, consultations, reports, labs, subscriptions, notifications), a shared layer (middleware, RBAC guards, utilities, error handling), and an infrastructure layer (database, Redis, Socket.IO, background queues, external services). This separation keeps business logic independent from framework and infrastructure details, and allows individual modules to be extracted into standalone microservices later without a full rewrite.

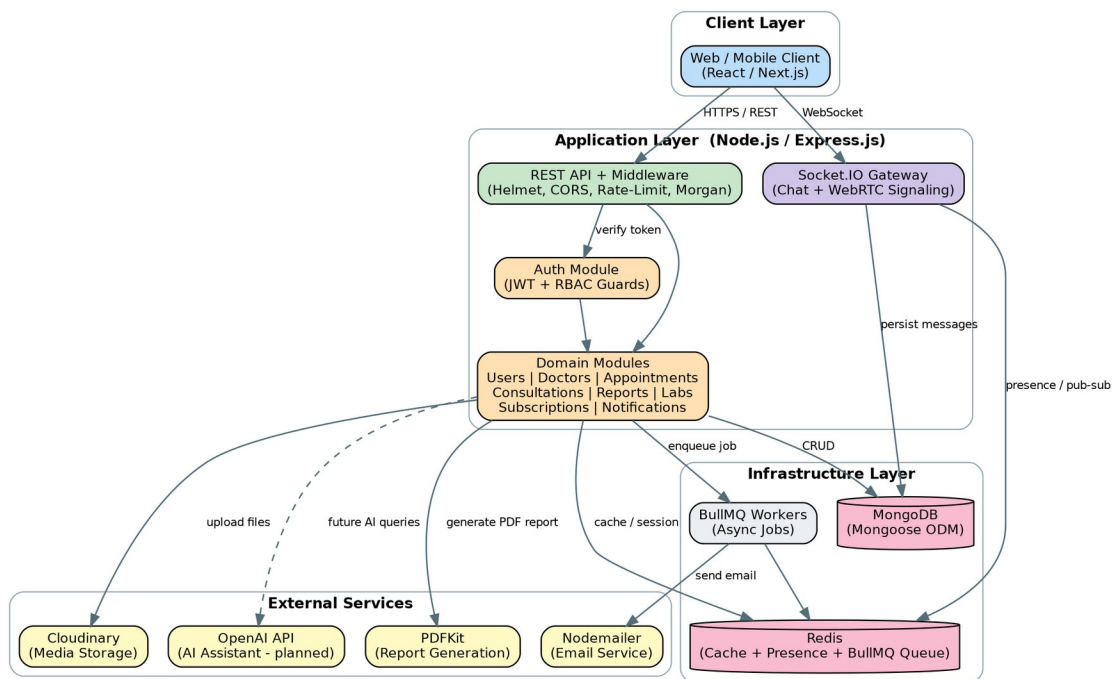


Figure 5.1 — High-level system architecture

5.1 Backend Folder Structure

- `modules/` — one folder per domain (auth, users, doctors, appointments, chat, video-call, consultations, reports, labs, subscriptions, notifications), each with its own routes, controller, service, and model.
- `shared/` — cross-cutting concerns: middleware, RBAC guards (auth/doctor/admin/lab), utilities (ApiError, catchAsync), and centralized error handling.
- `infrastructure/` — MongoDB connection, Redis client, Socket.IO server setup, BullMQ queues, mail, storage (Cloudinary), and PDF generation.
- `config/` — environment-based configuration for the database, Redis, Cloudinary, and Socket.IO.

5.2 Request Workflow Example — Booking an Appointment

The following flow illustrates how a typical write operation moves through the system, from the client to the database and back, including the supporting infrastructure it touches along the way.

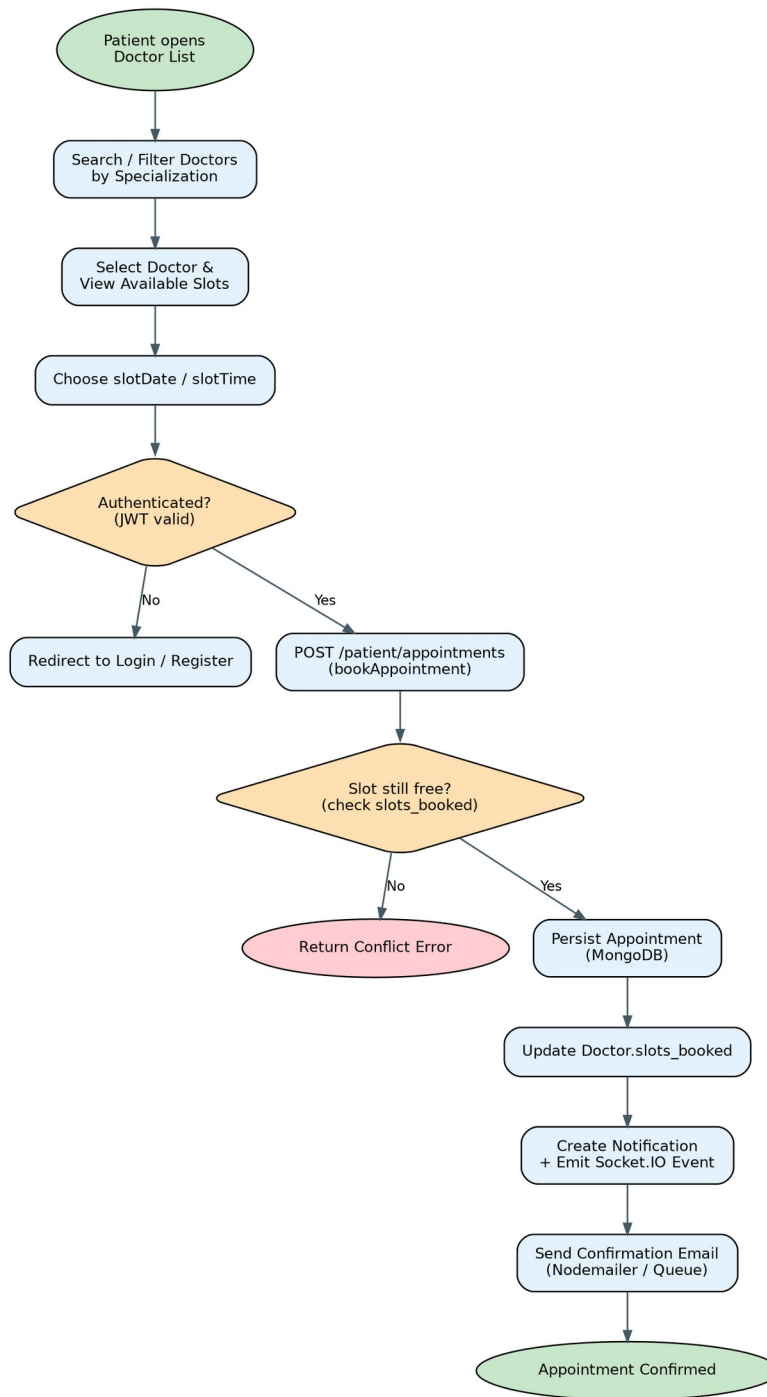


Figure 5.2 — Appointment booking workflow

5.3 Real-Time Communication Flow

Chat and video/audio consultation are handled outside the standard REST request/response cycle. When a patient sends a chat request, it is persisted as a ChatRequest document; once a doctor accepts it, both parties join a Socket.IO room identified by a deterministic roomId (userId_docId). Messages are emitted over that room and simultaneously persisted to MongoDB for history, while Redis tracks online/offline

presence. Video/audio calls follow the same room-based pattern: Socket.IO carries only the WebRTC signaling (offer/answer/ICE candidates) between the two peers, while the actual media stream flows directly between the browsers (peer-to-peer), and only call metadata (status, duration, timestamps) is stored in MongoDB.

6. Use Case Diagrams (UML)

Four actors interact with the system: Patient, Doctor, Admin, and Lab. Each use case below is derived directly from the implemented REST routes and is annotated with its type. Dashed arrows represent <<include>> relationships (a base use case always triggers another) and <<extend>> relationships (an optional use case that extends a base one under a condition).

Type	Meaning
<<Auth>>	Authentication-related use case (login/registration).
<<Core>>	Primary/base functional use case directly triggered by the actor.
<<Extend>>	Optional behavior that extends a base use case under a specific condition.
<<RealTime>>	Use case implemented over Socket.IO / WebRTC rather than plain REST.

6.1 Patient Use Case Diagram

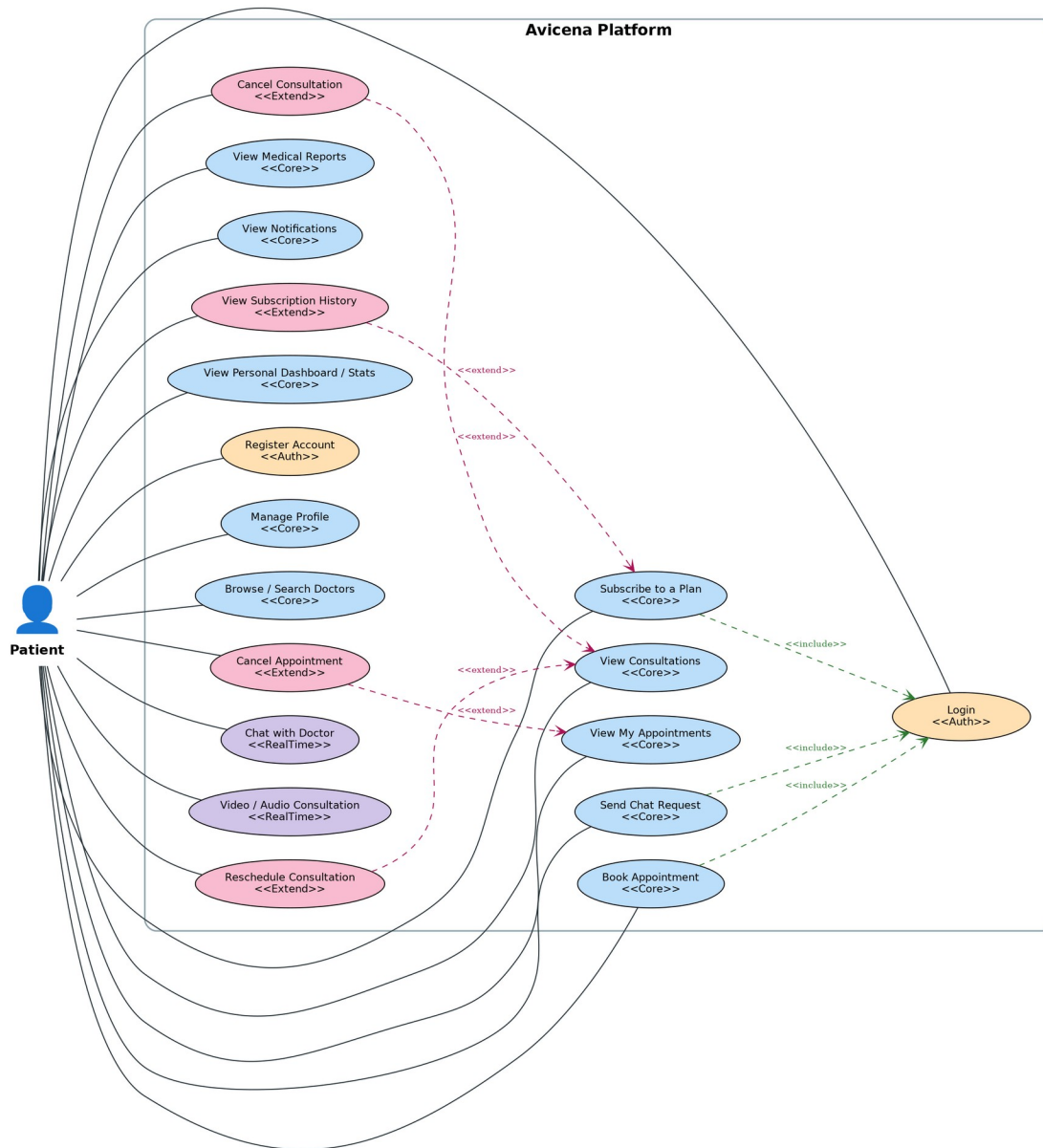


Figure 6.1 — Patient use cases

6.2 Doctor Use Case Diagram

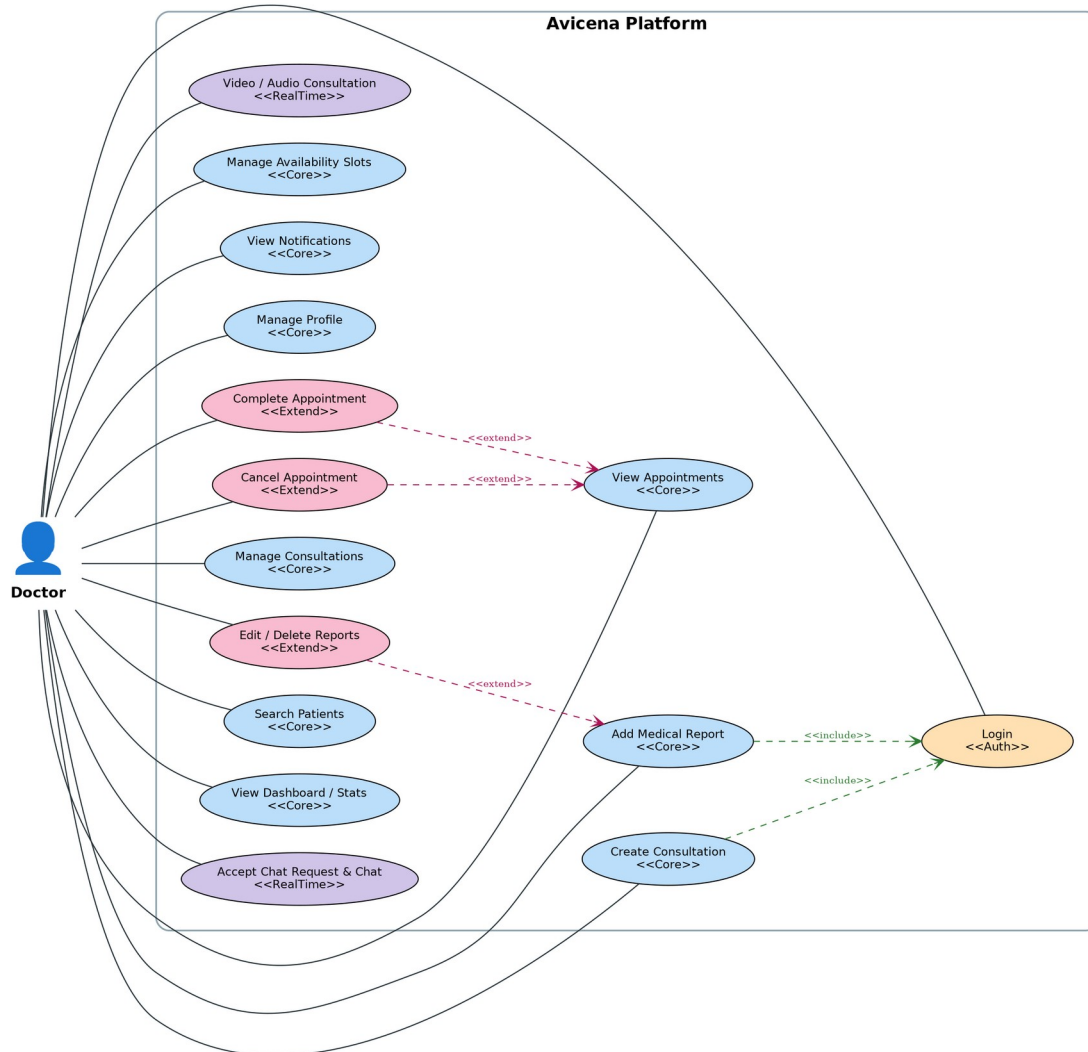


Figure 6.2 — Doctor use cases

6.3 Admin Use Case Diagram

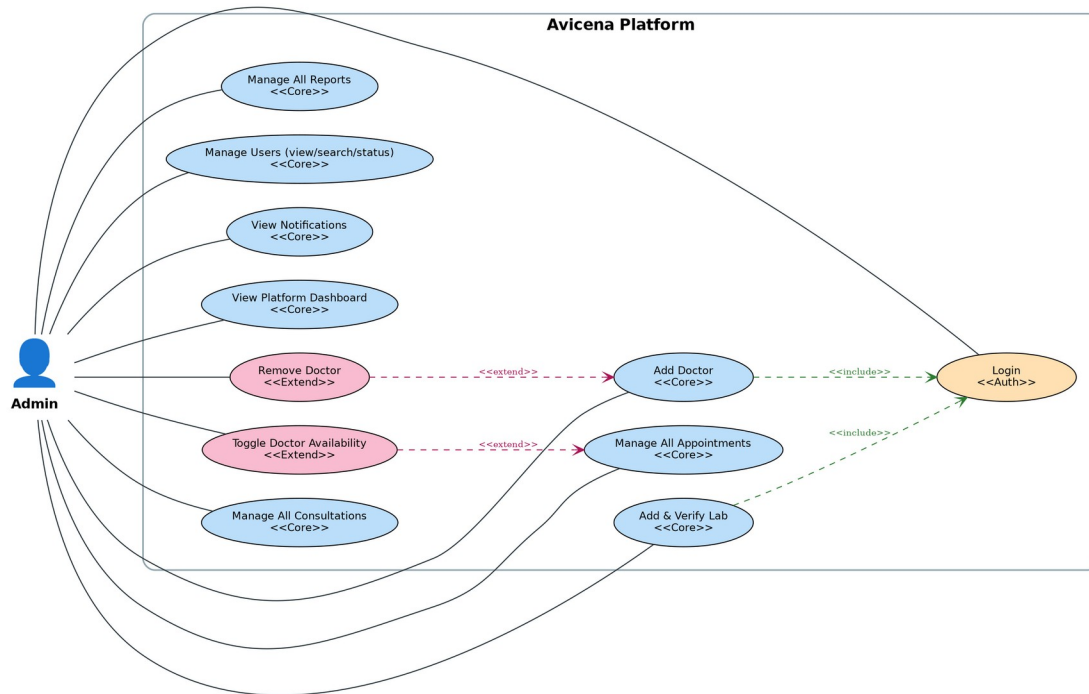


Figure 6.3 — Admin use cases

6.4 Lab Use Case Diagram

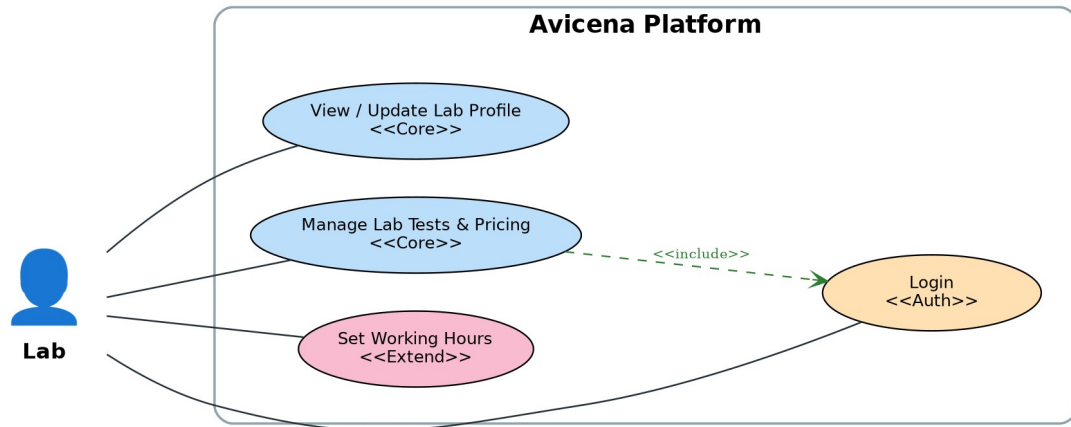


Figure 6.4 — Lab use cases

7. Database Design (Entity-Relationship Diagram)

Avicena uses MongoDB, a document database, so relationships below are modeled through referenced ObjectIds (foreign keys) rather than SQL joins, resolved at the application layer via Mongoose population. Several relationships are polymorphic (e.g., a Notification's recipient can be a User, Doctor, Admin, or Lab, distinguished by a recipientType discriminator field).

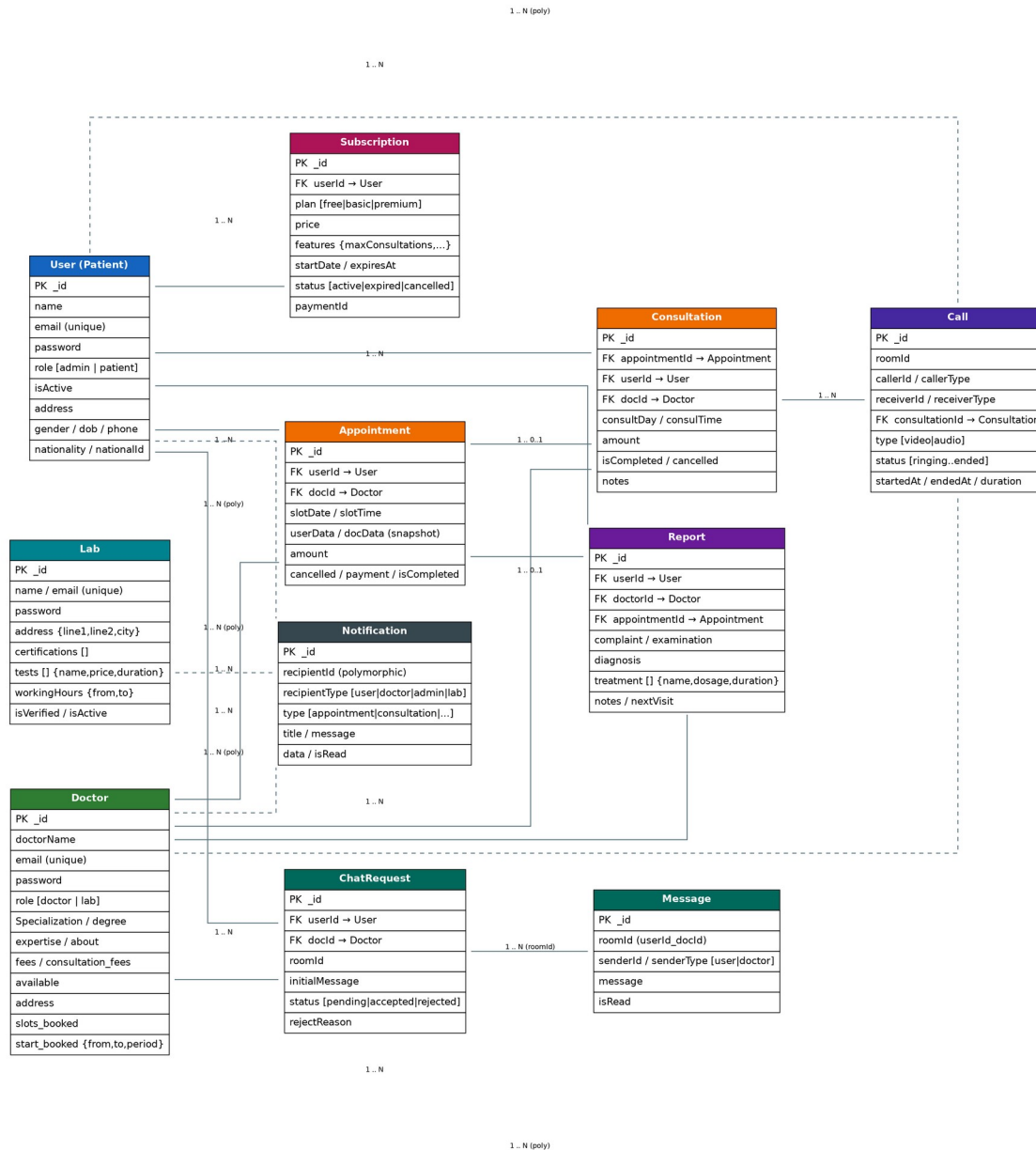


Figure 7.1 — Entity-Relationship Diagram

7.1 Core Entities

- **User** — patient accounts; also doubles as the Admin role via a role field.
- **Doctor** — doctor and lab-staff accounts; role field distinguishes doctor vs lab.

- Lab — laboratory profile, tests catalog, certifications, and working hours.
- Appointment — a booked slot between a User and a Doctor with a snapshot of both parties' data at booking time.
- Consultation — the clinical session tied to an Appointment, holding scheduling and completion state.
- Report — the medical record produced by a Doctor for a specific Appointment/patient (complaint, diagnosis, treatment).
- Subscription — a User's current or historical SaaS plan and feature entitlements.
- ChatRequest / Message — the handshake and message log for real-time chat between a User and a Doctor.
- Call — WebRTC session metadata linking two parties and, optionally, a Consultation.
- Notification — a polymorphic inbox entry addressed to any of the four actor types.

7.2 Editable Diagram Files

Alongside this document, fully editable diagram files are provided in draw.io / diagrams.net format (.drawio) for the ERD and all four Use Case diagrams, so they can be adjusted, re-branded, or extended directly.

8. Additional Details

8.1 Setup & Installation

The backend can be run locally with Node.js or fully containerized with Docker Compose (which also provisions MongoDB and Redis).

- Clone the repository: `git clone https://github.com/Mohamedali409/avicena-platform`
- Install dependencies: `npm install` (inside the Backend/ folder)
- Configure environment variables in a `.env` file (see 8.2)
- Run in development mode: `npm run dev` (or `docker compose up` for the full containerized stack)
- Seed sample data: `npm run seed`

8.2 Environment Variables

Variable	Purpose
PORT	Port the Express server listens on (default 5000/4000)
MONGO_URI / MONGODB_URL	MongoDB connection string
JWT_SECRET	Secret key used to sign and verify JWTs
REDIS_URL	Redis connection string (cache, presence, BullMQ)
EMAIL_SERVICE_KEY	Credential for the transactional email provider
ADMIN_EMAIL / ADMIN_PASSWORD	Legacy admin credential check used by the admin guard

8.3 Security Considerations

- Passwords are hashed (bcrypt/bcryptjs) before storage; raw passwords are never persisted.
- Helmet sets secure HTTP headers; express-rate-limit mitigates brute-force and abuse; CORS restricts cross-origin access.
- Each actor type has its own JWT scope and guard middleware, preventing privilege escalation across roles.

8.4 Project Roadmap

- Phase 1 (MVP): Authentication, doctor & patient management, appointment system.
- Phase 2: Real-time chat, notifications, file/report uploads.
- Phase 3: Video consultation, subscription system.
- Phase 4: Labs integration, admin dashboard, analytics.
- Phase 5 (Advanced): AI chatbot assistant, smart recommendations, React Native mobile app.

8.5 Project Type & Purpose

Avicena is positioned as a portfolio / graduation-project simulation of a real-world SaaS telemedicine platform, deliberately built with production-grade patterns (RBAC, modular architecture, queues, caching, containerization) rather than a minimal class-project scope.

8.6 Conclusion

Avicena demonstrates a complete, realistic approach to building a healthcare SaaS product: a clearly separated modular backend, real-time communication layered on top of a conventional REST API, and a data model designed for extensibility toward labs, subscriptions, and eventually AI-assisted care. The architecture choices — modular monolith with a clean path to microservices, Redis-backed real-time presence, and queue-based background processing — make the codebase a solid foundation for continued development beyond the current MVP phase.